

Python Programming

Comprehensive Concept Notes: Units 1 & 2

UNIT I: Installation, Data Types and Input/Output

► 1. Features, Importance & Installation

- **Features of Python:**

- **Simple and Easy to Learn:** Clean, English-like syntax without semicolons or curly braces makes it highly readable.
- **Dynamically Typed:** Variables do not need explicit type declarations; types are determined at runtime.
- **Interpreted Language:** Code is executed line-by-line, which makes debugging faster and easier.
- **Platform Independent:** Follows "Write Once, Run Anywhere"; works seamlessly across Windows, macOS, and Linux.
- **Extensive Standard Library:** Comes with "batteries included," providing built-in modules for regex, mathematics, network programming, and more.
- **Multi-paradigm:** Supports Object-Oriented, Procedural, and Functional programming styles.

- **Importance:** Created by Guido van Rossum, Python is widely used in cutting-edge fields like Data Science, Artificial Intelligence (AI/ML), Web Development, and

Scripting/Automation due to its massive community support and ease of use.

- **Windows Installation:** Download the .exe from python.org. **Crucial Step:** Check "Add Python to PATH" during setup so it can be run from the command prompt.
- **Ubuntu Installation:** Python 3 is usually pre-installed. To update/install: run `sudo apt update` followed by `sudo apt install python3`.

▶ 2. Executing Programs & Internal Working

- **Interactive Mode (REPL):** Type `python` in terminal. Code executes line-by-line instantly. Good for testing quick logic.
- **Script Mode:** Code is written in a file (e.g., `app.py`) and executed using `python app.py`.

Internal Working of Python: Python is an interpreted language, but it utilizes a hidden compilation step.

Source Code (.py) → Compiler → Bytecode (.pyc) → Python Virtual Machine (PVM) → Machine Code

The PVM reads the bytecode line-by-line and executes it on the host operating system.

▶ 3. Character Set, Tokens & Comments

- **Character Set:** Python uses the Unicode character set (A-Z, a-z, 0-9, special symbols, whitespaces).
- **Tokens:** The smallest individual unit in a Python script.
 1. **Keywords:** Reserved words (e.g., `if`, `def`, `True`, `class`). Cannot be used as variable names.
 2. **Identifiers:** User-defined names. *Rules:* Must start with a letter or underscore `_`, cannot contain special characters (like `@` or `$`), case-sensitive (`Age` vs `age`).
 3. **Literals:** Raw data values assigned to variables (e.g., `100`, `"Hello"`).

4. **Operators:** Symbols that trigger computation (`+` , `=`).

5. **Punctuators:** Structural symbols used to organize code (`,` , `:` , `()` , `[]`).

- **Comments:** Used for code documentation. Ignored by the PVM.

Single-line: `# Note here` . Multi-line (Docstrings): `""" Notes here """` .

► 4. Variables & Python Core Data Types

Variables act as labels pointing to memory locations. Python uses **dynamic typing**; you don't declare the type (e.g., `x = 10` automatically makes `x` an integer). Multiple assignment is supported: `a, b = 5, 10` .

Category	Data Types	Mutability (Can it be changed?)
Numeric	<code>int</code> (10), <code>float</code> (3.14), <code>complex</code> (2+3j)	Immutable
Sequence	<code>str</code> ("Text"), <code>tuple</code> ((1, 2))	Immutable
Sequence	<code>list</code> ([1, 2, "a"])	Mutable
Mapping	<code>dict</code> ({"key": "value"})	Mutable
Sets	<code>set</code> ({1, 2, 3})	Mutable
Boolean	<code>bool</code> (True, False)	Immutable

► 5. Input and Output Functions

- **print() function:** Outputs data. Has useful parameters:
 - `sep` : Separator between values (default is space).
 - `end` : What to print at the end (default is newline `\n`).

```
print("Apple", "Banana", sep=" | ", end="!!!")  
# Output: Apple | Banana!!!
```

- **input() function:** Halts execution to take user input. **Always returns a string**, requiring typecasting for math.

```
age = int(input("Enter age: ")) # Typecasted to integer
```

- **eval() function:** Parses a string expression and evaluates it as Python code. Highly useful for getting dynamic data types directly from the user.

```
result = eval("10 * 5") # result becomes 50 (int)  
user_list = eval(input("Enter a list: ")) # If user types [1,2], it becomes an actual
```

► 6. Strings in Python

A string is an **immutable** sequence of characters. They are one of the most widely used data types for text manipulation.

- **Creation:** Enclosed in single `'...'`, double `"..."`, or triple `'''...'''` quotes (triple quotes are used for multi-line strings or docstrings).
- **Indexing:** Accessing individual characters. Python supports *positive indexing* (left-to-right, starting at 0) and *negative indexing* (right-to-left, starting at -1).

```
s = "PYTHON"  
print(s[0]) # Output: P  
print(s[-1]) # Output: N (last character)
```

- **Slicing (Positive & Negative):** Extracting a specific portion (substring) using the syntax `string[start:stop:step]`. The `stop` index is always exclusive.
 - **Positive Slicing:** Uses standard left-to-right indexing.

- **Negative Slicing:** Uses right-to-left indexing, very useful for extracting from the end of a string without knowing its full length.

```
msg = "Hello World"

# Positive Slicing
print(msg[0:5])    # Output: Hello (indexes 0 to 4)
print(msg[6:])      # Output: World (from index 6 to the end)

# Negative Slicing
print(msg[-5:])     # Output: World (Starts from 5th character from the end)
print(msg[-5:-1])  # Output: Worl (Excludes the very last character 'd')

# Step / Reverse
print(msg[::-1])    # Output: dlroW olleH (A trick to reverse the string)
```

UNIT II: Operators and Control Statements

► 1. Operators in Python

- **Arithmetic:** `+` (Add), `-` (Sub), `*` (Mult), `/` (Float Div), `//` (Floor Div - removes decimals), `%` (Modulo - returns remainder), `**` (Exponent/Power).
- **Compound Assignment:** A shorthand. `x += 5` expands to `x = x + 5`. Works with `-=`, `*=`, etc.
- **Bitwise Operators:** Operates directly on the binary representations of integers.
 - `&` (AND): Sets bit to 1 if both are 1.
 - `|` (OR): Sets bit to 1 if one of two is 1.
 - `^` (XOR): Sets bit to 1 if only one is 1.

- `~` (NOT): Inverts all the bits.
- `<<` / `>>` : Left / Right Shift. Shifts bits, effectively multiplying/dividing by 2.
- **String Operators (Concatenation & Repetition):** In Python, the `+` and `*` operators are overloaded to work with sequence data types like strings.
 - **Concatenation (`+`):** Joins two or more strings together end-to-end.
`"Hello" + " " + "World"` results in `"Hello World"` .
 - **Repetition (`*`):** Repeats a string a specified number of times.
`"Ha" * 3` results in `"HaHaHa"` .

Operator Precedence & Associativity:

When an expression has multiple operators (e.g., `5 + 3 * 2`), Precedence determines which executes first (Multiplication before Addition, so 11). If precedence is the same (e.g., `10 / 2 * 5`), Associativity decides the direction (Left-to-Right, so 25). Note: Exponentiation `**` is Right-to-Left associativity.

► 2. Decision Statements & Boolean Logic

- **Relational (Comparison) Operators:** Compares two values. Returns `True` or `False` . (`==` , `!=` , `>` , `<` , `>=` , `<=`).
- **Boolean Operators (Logical):** Combines conditional statements.
`and` (True if both are True), `or` (True if at least one is True), `not` (Reverses the result).
- **Boolean Expressions:** Any expression that evaluates to a boolean value. E.g., `(x > 5)` and `(y < 10)` .
- **Decision Making Statements:** Controls the flow using `if` , `elif` , and `else` . Indentation (usually 4 spaces) defines the blocks.

```
# Example of a nested if-else with Boolean operators
age = 25
has_ticket = True

if age >= 18 and has_ticket:
```

```
if age >= 65:
    print("Senior Entry Allowed")
else:
    print("Standard Adult Entry Allowed")
else:
    print("Entry Denied")
```

► 3. Loop Control Statements

Loops are used to execute a block of code repeatedly.

- **The while loop:** Executes continuously as long as a Boolean condition remains True. You must manually update the iterator to prevent infinite loops.

```
count = 0
while count < 3:
    print("Count is", count)
    count += 1 # Crucial: Incrementing the counter
```

- **The range() function:** Generates a sequence of immutable numbers. Syntax: `range(start, stop, step)` . Note that 'stop' is exclusive.
Example: `range(1, 10, 2)` generates 1, 3, 5, 7, 9.
- **The for loop:** In Python, `for` loops act as iterators over sequences (like lists, strings, or a range).

```
# Iterating using range
for i in range(1, 4):
    print(i) # Prints 1, 2, 3

# Iterating over a string (Sequence)
for char in "Python":
    print(char) # Prints P, y, t, h, o, n line by line
```

► 4. Jump Statements

Used to alter the normal execution flow of loops.

Statement	Behavior	Example Code Concept
<code>break</code>	Exits/terminates the current loop completely, skipping any remaining iterations.	<pre>if item == "found": break</pre> (Stop searching)
<code>continue</code>	Skips the rest of the code inside the current loop iteration and forces the <i>next iteration</i> to begin.	<pre>if num < 0: continue</pre> (Skip negative numbers, process next)